Materi Praktik Aplikasi Mobile Lanjut

Topik: Membuat Aplikasi Form Login Menggunakan Flutter dan Firebase

---

1. Capaian Pembelajaran (Learning Outcomes)

Setelah mengikuti praktik ini, mahasiswa mampu:

1. Memahami konsep autentikasi pada aplikasi mobile.
2. Mengintegrasikan Flutter dengan Firebase.
3. Membuat form login dan register dengan validasi.
4. Mengimplementasikan Firebase Authentication (Email & Password).
5. Menangani error autentikasi.
6. Mengarahkan user ke halaman dashboard setelah login berhasil.
7. Menerapkan best practice UI sederhana untuk form autentikasi.

---

2. Deskripsi Singkat

Autentikasi merupakan komponen penting dalam aplikasi modern untuk memastikan keamanan data pengguna. Pada praktik ini, mahasiswa akan membangun aplikasi login menggunakan **Flutter** sebagai frontend dan **Firebase Authentication** sebagai backend tanpa harus membangun server sendiri.

---

3. Tools dan Software

Wajib:

- Flutter SDK (≥ 3.x)
- Android Studio / VS Code
- Firebase Account
- Emulator atau perangkat Android
- Dart SDK

Opsional:

- Git

---

4. Arsitektur Sistem

Alur Sistem:

User → Form Login Flutter → Firebase Authentication →
✔ Berhasil → Dashboard
✖ Gagal → Pesan Error

---

5. Langkah Praktik

✅ Praktik 1 — Membuat Project Flutter Baru

```
flutter create login_firebase_app
cd login_firebase_app
flutter run
```

✅ Praktik 2 — Membuat Project Firebase

Langkah:

1. Buka:
   👉 https://console.firebase.google.com
2. Klik **Create Project**
3. Tambahkan aplikasi Android dengan package name:

```
com.example.loginfirebase
```

4. Download file:

```
google-services.json
```

5. Masukkan ke folder:

```
android/app/
```

✅ Praktik 3 — Tambahkan Dependency

Edit **pubspec.yaml**

```
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^2.30.0
  firebase_auth: ^4.17.0
```

Lalu jalankan:

```
flutter pub get
```

✅ Praktik 4 — Konfigurasi Firebase di Flutter

Edit **main.dart**

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'login_page.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: LoginPage(),
    );
  }
}
```

✅ Praktik 5 — Aktifkan Authentication

Masuk ke Firebase Console:

👉 Authentication
👉 Sign-in Method
👉 Enable **Email/Password**

✅ Praktik 6 — Membuat UI Login

Buat file:

`login_page.dart`

Fitur UI:

- ✔ Icon user
- ✔ TextField email
- ✔ TextField password
- ✔ Tombol login
- ✔ Tombol ke register

Contoh Coding Login UI

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'dashboard_page.dart';

class LoginPage extends StatefulWidget {
  @override
  _LoginPageState createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {

  final emailController = TextEditingController();
  final passwordController = TextEditingController();

  Future login() async {
    try {
      await FirebaseAuth.instance.signInWithEmailAndPassword(
        email: emailController.text.trim(),
        password: passwordController.text.trim(),
      );

      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (_) => DashboardPage()),
      );

    } on FirebaseAuthException catch (e) {
      ScaffoldMessenger.of(context)
          .showSnackBar(SnackBar(content: Text(e.message!)));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
```

```
        child: Padding(
          padding: EdgeInsets.all(25),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [

              Icon(Icons.account_circle, size: 100),

              SizedBox(height: 20),

              TextField(
                controller: emailController,
                decoration: InputDecoration(
                  labelText: "Email",
                  border: OutlineInputBorder(),
                ),
              ),

              SizedBox(height: 15),

              TextField(
                controller: passwordController,
                obscureText: true,
                decoration: InputDecoration(
                  labelText: "Password",
                  border: OutlineInputBorder(),
                ),
              ),

              SizedBox(height: 20),

              ElevatedButton(
                onPressed: login,
                child: Text("Login"),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

✅ Praktik 7 — Membuat Dashboard

```
dashboard_page.dart
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'login_page.dart';

class DashboardPage extends StatelessWidget {

  void logout(BuildContext context) async {
    await FirebaseAuth.instance.signOut();

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => LoginPage()),
```

```
    );
  }

  @override
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;

    return Scaffold(
      appBar: AppBar(
        title: Text("Dashboard"),
        actions: [
          IconButton(
            icon: Icon(Icons.logout),
            onPressed: () => logout(context),
          )
        ],
      ),
      body: Center(
        child: Text(
          "Selamat datang ${user?.email}",
          style: TextStyle(fontSize: 18),
        ),
      ),
    );
  }
}
```

✅ Praktik 8 — Membuat Register Page (Opsional tetapi Disarankan)

Gunakan method:

```
FirebaseAuth.instance.createUserWithEmailAndPassword(
  email: email,
  password: password,
);
```

---

6. Validasi Form (Best Practice)

Gunakan:

- Minimal password: **6 karakter**
- Format email valid
- Jangan kosong

Disarankan memakai:

```
Form + GlobalKey<FormState>
```

---

7. Handling Error Firebase

Contoh error umum:

Error	Penyebab
user-not-found	Email tidak terdaftar
wrong-password	Password salah
email-already-in-use	Email sudah digunakan

---

8. Keamanan (Advanced Insight untuk Mahasiswa Lanjut)

Ajarkan mahasiswa:

❗ Jangan simpan password di database sendiri.

Firebase sudah menggunakan:

- Hashing
- Token authentication
- Secure session

---

9. Latihan Mahasiswa

🔥 Latihan 1 (Wajib)

Tambahkan:

✔ Halaman Register
✔ Konfirmasi password

🔥 Latihan 2 (Menengah)

Buat fitur:

✔ Reset password
Gunakan:

```
FirebaseAuth.instance.sendPasswordResetEmail(email: email);
```

🔥 Latihan 3 (Advanced)

Buat:

✔ Auto login jika user masih tersimpan
Gunakan:

```
FirebaseAuth.instance.authStateChanges()
```

---

10. Penilaian Praktik

Komponen	Bobot
UI Login	20%
Integrasi Firebase	25%
Validasi Form	15%
Navigasi	15%
Error Handling	15%
Kerapian Kode	10%

---

11. Kesalahan Umum Mahasiswa

Biasanya terjadi pada:

- Salah package name
- Lupa initialize Firebase
- Tidak enable Email/Password
- Password < 6 karakter

---

12. Pengembangan Lanjutan (Sangat Direkomendasikan untuk Mata Kuliah Lanjut)

Topik berikut bisa menjadi **pertemuan selanjutnya**:

✅ Login dengan Google
✅ Login dengan biometrik
✅ Role-based authentication (admin/user)
✅ Firebase Firestore integration
✅ Clean Architecture
✅ State Management (Provider / Riverpod / Bloc)

---

13. Referensi

1. Flutter Documentation
   https://docs.flutter.dev
2. Firebase Documentation
   https://firebase.google.com/docs/auth
3. Firebase Auth for Flutter
   https://firebase.flutter.dev